Emergent Complexity: Initial Assignment

Andrew Kim

September 9, 2026

Website https://wind3264.github.io/emergent-complexity/
Repository https://github.com/wind3264/emergent-complexity

1 What I built and how it works

Life Lab is an interactive laboratory for binary outer-totalistic cellular automata on the Moore neighbourhood. It is a static Next.js and TypeScript site, deployed to GitHub Pages by a GitHub Actions workflow that runs the test suite first, so the published site is always a build that passed its tests.

The site lets you draw cells, run, pause and single-step, set the speed from 1 to 120 generations per second, seed random soups at any density, resize the board up to 400×260, toggle between a torus and dead edges, stamp catalogued patterns, edit the rule, and inject noise. A footer reports generation, population, density, a population sparkline, and an automatic verdict when the board goes extinct, freezes, or falls into a short cycle.

The engine. `src/lib/life.ts` is the whole simulator and has no dependency on React. A rule is two 9-bit masks, so applying it is one shift and one bitwise AND: bit n of `birth` says whether a dead cell with n live neighbours turns on, and bit n of `survive` says whether a live cell with n live neighbours stays on. That is exactly the 2^{18} rule space the assignment describes. A board is a flat `Uint8Array`, and each generation is written into a second buffer that is then swapped in, so nothing is allocated per step. Wrapping is resolved once per row and column rather than once per neighbour.

The rule editor. The eighteen bits are exposed as two rows of nine toggles. Each toggle draws the situation it controls: a 3×3 neighbourhood with a dead or live centre and exactly n neighbours lit. Toggling rewrites the rule string immediately, and typing a rule string or choosing one of thirteen presets moves the toggles.

Rendering. The simulation runs in a `requestAnimationFrame` loop that reads refs and paints the canvas directly, so a board running at 120 generations per second never re-renders React; only the statistics footer updates, at about 10 Hz. Cells born this generation are painted brighter than surviving cells, which is what makes gliders and growth fronts visible at a glance, and cells flipped by noise are painted a third colour so an injected perturbation can be told apart from something the rule did.

Experiments. Everything quantitative in this report comes from scripts in `scripts/` that import the same engine the website runs, so a number in a table and a number on the screen come from identical code. Every script uses a seeded generator, so re-running it reproduces the tables exactly. `npm test` runs 31 tests over the engine and the pattern catalogue, including assertions that each catalogued pattern does what its description claims.

2 What I investigated

1. What does a random Conway soup turn into, and how much does the initial density matter?
2. What is left on the board once it settles, and in what proportions? (Tasks 1 and 2)
3. What do randomly sampled rules do, and how much of the 2^{18} space is worth looking at?
4. Can a rule as interesting as Conway's be found by searching rather than by sampling, and what would the search criterion have to be? (Task 3)
5. What happens when cells are flipped at random during the simulation: which structures are robust, which are not, and is there a noise level at which the behaviour changes character rather than merely degrading? (Task 4, option B)

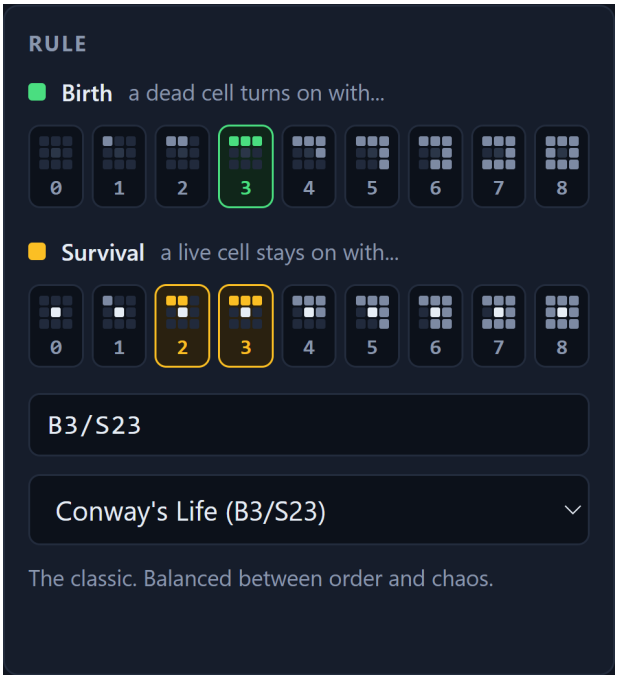
3 Tasks 1 and 2: Conway's Life and the rule editor

These two tasks are written up in full in `docs/observations.md`; this is the short version. Each trial fills a 96×96 torus at a chosen density and runs until the board exactly repeats a state it has already been in, which means it has entered a cycle, or until 3000 generations pass. Repeats are found by hashing every generation, so a cycle of any length up to the cap is detected and the generation it starts at is known exactly.

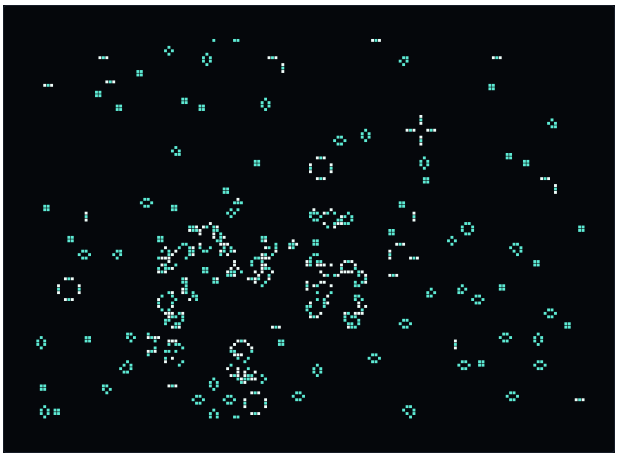
The initial density barely matters. Twelve soups at each of fourteen densities from 5% to 80% under `B3/S23` all settle to roughly the same place. Over the whole range from 10% to 70% the mean final density stays between 2.6% and 3.1%, while the initial density varies by a factor of seven. Life has an attractor, and the starting density mostly chooses how long the board takes to get there, not where it lands. The two ends behave differently for opposite reasons: at 5% the soup is too sparse and collapses within about 19 generations, and at 80% almost every cell dies of overcrowding in the first step or two.

What is left over is a short list of shapes. Taking every isolated structure on 13 settled Conway boards, 590 in total, five shapes account for 95% of them: block 41.5%, beehive 23.2%, blinker 17.6%, boat 6.9%, loaf 6.4%. About 82% of the leftovers are still lifes that never change again, and essentially all of the rest are period-2 blinkers. Gliders are real but rare: about 8% of soups still contain one after 1500 generations, and a single glider raises the recurrence period of the whole board from 2 to 384, which is 4×96, the time it takes to cross the torus and return.

HighLife. `B36/S23` is one click from Conway in the editor. Its statistics are close but consistently lower, about 2.0% mean final density against 2.8%, and it settles more reliably: all 168 HighLife runs reached a cycle within 3000 generations while 19 of the 168 Conway runs did not. The ash is made of the same objects in different proportions, because none of the common Conway still lifes ever presents a cell with exactly six live neighbours. The genuinely new object is the 12-cell replicator, which becomes two translated copies of itself every 12 generations; counting copies at multiples of 12 gives 1, 2, 2, 4, 2, 4, which are the odd entries in successive rows of Pascal's triangle, so the copies over time draw a Sierpinski triangle.



(a) The rule editor. Each toggle draws the neighbourhood situation it controls, so the eighteen bits of the rule are visible rather than encoded in a string.



(b) A Conway board after about 1400 generations: sparse islands of a few repeated shapes, separated by empty space that nothing in the rule ever mentions.

Figure 1: The simulator at the end of Task 2.

4 Task 3: what most of the rule space does

4.1 Method

Each rule is run from five random soups on a 64×64 torus at densities 5%, 15%, 30%, 50% and 75%, for 800 generations, and four things are measured.

- **Density**, the fraction of live cells, averaged over the last 50 generations.
- **Activity**, the fraction of cells that change state in a generation, averaged over the same window. This is the quantity that separates a board frozen into still lifes, which has activity zero however full it is, from one that is still boiling.
- **Transient length**, the generation at which the board first repeats a state it has already been in, found by hashing every generation.
- **Self-reproduction**, tested directly: a 5-cell seed is run alone and the board is checked at every generation for being exactly two or more disjoint translated copies of that seed and nothing else. The copies are peeled off greedily, which is exact rather than a heuristic, because the topmost-then-leftmost live cell has to be the topmost-then-leftmost cell of some copy.

Rules are then labelled by cuts through density and activity. The cuts are arbitrary, but they are stated, reproducible, and applied identically to every rule: extinction below 0.2% density, saturating above 50%, frozen below 0.2% activity, locally oscillating below 2%, chaotic above 12%, and complex in between.

4.2 Result 1: most of the rule space does one of two boring things

class	rules	mean density	mean activity
extinction	1	0.00%	0.00%
frozen	6	5.27%	0.04%
locally oscillating	6	20.50%	0.69%
complex	1	49.96%	8.38%
chaotic	36	39.89%	50.21%
saturating	49	64.01%	41.57%
self-replicating	1	50.48%	49.52%

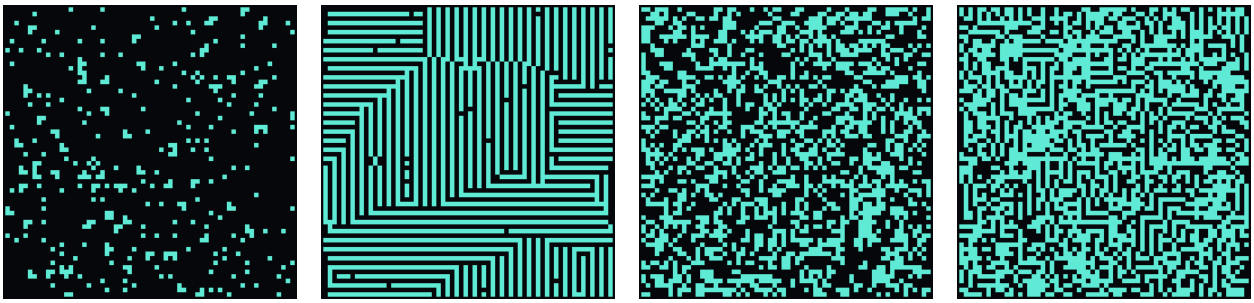
Table 1: 100 rules sampled uniformly from the 2^{17} rules without B0. 85 of the 100 either fill the board or boil.

Eighty-five of a hundred rules either saturate, meaning the board settles above half full, or stay chaotic, meaning around 40% of cells are changing state every generation forever. Only 13 do anything sparse and structured, and those are mostly frozen. Conway's rule is not a typical point in its own rule space, and it is not close to one.

One rule in the sample, `B1357/S013578`, passed the self-reproduction test: a 5-cell seed run alone becomes two disjoint translated copies of itself and nothing else. It is a real replicator, found by random sampling, but it replicates into a board that is half full and boiling, so the copies are immediately destroyed by their surroundings.

Half the rule space is spoiled by one bit. The first sample was drawn uniformly from all 2^{18} rules, and 51 of the 100 contained B0, meaning a dead cell with no live neighbours turns on. An empty region under such a rule fills instantly, so the entire background of the board flashes on at the first step and every one of these rules is explosive from the start. That single bit accounts for half of the rule space and collapses all of it into essentially one behaviour, which is why the main sample above excludes it: uniform sampling over 2^{18} mostly measures B0.

One rule is not one behaviour. 17 of the 100 rules were labelled differently depending on which soup they started from. The commonest disagreement is frozen against locally oscillating (4 rules), but three rules went extinct from one initial density and saturated from another, which is as far apart as two outcomes can be. A single simulation is not evidence about a rule.



frozen locally oscillating chaotic saturating

Figure 2: One representative of each of the four commonest classes, at generation 800 from a 30% soup on a 64×64 torus.

4.3 Result 2: searching instead of sampling

The sample contains nothing worth a closer look, which the assignment anticipates, so I searched instead. The search is two stages, and its criterion comes from Task 1 rather than from taste.

Stage one screens 3000 rules without B0 on a single 48×48 soup at 30% density, keeping those that end between 0.5% and 15% full and are still changing at least 0.2% of their cells per generation. A rule is abandoned at generation 60 if it is already more than 35% full, which is what makes screening thousands of rules affordable: most of the space fills the board within a few dozen generations and can be rejected there. **87 of 3000 rules survived, 2.9%.**

Stage two runs the survivors through the full five-soup measurement and keeps those with the two properties Conway turned out to have in Task 1: *every* soup settles between 0.2% and 12% full, so the final density is an attractor rather than a reflection of the starting density, and the board is still changing between 0.1% and 5% of its cells per generation. 51 of the 87 passed. The survivors are ranked by mean transient length, on the grounds that a rule whose soups take hundreds of generations to resolve is doing more on the way than one that freezes in ten.

The lower end of that density band is calibrated on Conway, and this is worth admitting: my first version used a floor of 0.5%, and it rejected Conway's own rule, because Conway's 5% soup collapses to 0.44% full. A filter that rejects the one rule known in advance to be worth finding is a broken filter, not a strict one.

Mobility is then tested directly rather than inferred. Every pattern that fits in a 4×4 box, one per shape up to rotation and reflection, 7624 of them, is run alone under each surviving rule until it dies, repeats where it stands, or reappears in the same shape somewhere else. The test finds the glider under `B3/S23`, 5 cells and period 4, which is the check that it works.

rule	density	activity	spaceship shapes	score (mean transient)
`B368/S0136`	4.92%	0.47%	0	207
`B367/S0138`	5.24%	0.61%	0	176
`B357/S0248`	2.15%	0.44%	0	160
`B37/S01368`	5.08%	0.52%	0	156
`B358/S126`	2.39%	0.24%	**3**	117
`B3/S23` (Conway)	2.41%	1.10%	2	320
`B36/S23` (HighLife)	2.79%	1.18%	2	553

Table 2: The top of the search, with the two known rules measured the same way for calibration.

Two things stand out. First, the search works: 51 rules have Conway's macroscopic behaviour, and several are sparser and more regular than Conway is. Second, none of them beats Conway on the thing the score actually measures. Conway's transients last 320 generations on this board and HighLife's 553, against 207 for the best rule found. Reproducing the statistics of Conway's steady state turns out to be much easier than reproducing how long it takes to get there.

The sharpest result is mobility: **only 3 of the 51 rules have a spaceship at all.** Sparse, slow and structured is not rare. Sparse, slow, structured *and* able to move information across the board is rare.

4.4 A closer look at `B358/S126`

I chose the best-scoring rule that also has a spaceship. It is a cleaner version of Conway's attractor, and a much poorer one in every other respect.

- **The attractor is tighter than Conway's.** Across starting densities from 10% to 80% the

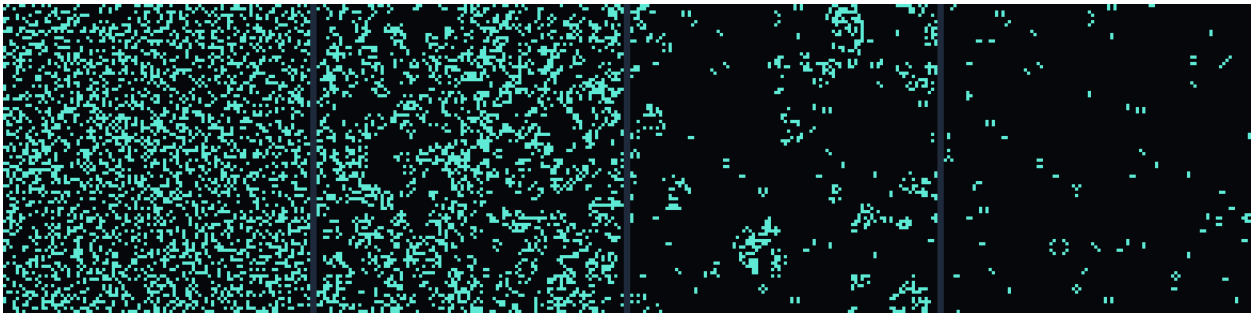


Figure 3: `B358/S126` on a 96×96 torus at generations 0, 20, 100 and 800 from a 30% soup. The board thins out monotonically and ends as isolated dominoes and diagonal pairs.

	B358/S126			B3/S23
initial density	transient	period	final density	final density
5%	19	12	1.61%	0.44%
15%	105	12	2.56%	2.68%
30%	154	12	1.93%	4.21%
50%	151	12	3.25%	4.25%
75%	154	12	2.61%	4.29%

Table 3: Every soup lands in the same narrow band and every one ends in a period-12 cycle. Conway's figures are from a separate sweep capped at 1200 generations, where many boards have not finished settling; its fully settled value is about 2.8%.

final density stays between 2.1% and 2.5%. Conway varies more, and unlike Conway this rule does not collapse at 80%: a very full board thins out to the same 2% rather than dying of overcrowding.

- **Everything has the same period.** All five soups end in a cycle of period 12, and every oscillator found in the ash is period 12 as well. Conway's ash is period 1 and 2 with the occasional oddity; this rule has one clock and everything runs on it.
- **The vocabulary is poorer.** Conway's ash is dominated by the block, a shape with internal structure. This rule leaves mostly 2-cell dominoes and diagonal pairs, 94 of the 105 isolated structures found. There is nothing here like the loaf or the pond.
- **It does have a spaceship**, 6 cells with period 6, travelling 2 cells straight, so speed $c/3$ orthogonally rather than Conway's $c/4$ diagonally. It is in the site's pattern menu as *B358 spaceship*, and the rule is in the preset list as *Sparse (searched)*.

```
#..#
 .##.
 ##..
```

What this comparison actually taught me is that "looks like Life" decomposes into properties that can be had separately. `B358/S126` has Life's attractor, more cleanly than Life does. It does not have Life's long transients, its variety of stable shapes, or anything like the density of interesting objects. Sparseness is cheap; richness is not.

5 Task 4, option B: noise and perturbations

5.1 What I added, and the choices behind it

Every cell is flipped independently with probability p after each generation, on or off, regardless of the rule. The site exposes p on a logarithmic slider from off to 5×10^{-2}, and reports how many of the board's cells that works out to per generation, because $p = 10^{-3}$ means nothing until you know it is 18 cells on a 160×110 board.

Three implementation choices are worth stating.

Symmetric flips, not injections. A cell that is on can be turned off and a cell that is off can be turned on, with the same probability. This is the reading of the assignment's "randomly turn cells on or off", and it is the choice with the fewest parameters, but it is not neutral: a Conway board is about 97% dead, so in practice almost every flip is a spontaneous birth. A model with separate on and off rates would be a different experiment, and Section 6 lists it as a limitation.

Sampling the gaps, not the cells. Drawing one random number per cell would cost 104,000 random numbers per generation on the largest board, which is more work than the rule itself. Instead `applyNoise` draws the gap to the next flipped cell straight from the geometric distribution that governs it, so the cost is proportional to the number of flips rather than to the size of the board. It is exact, not an approximation, and it is asserted in the tests.

Noise turns the cycle detector off. The site normally announces when a board freezes or falls into a cycle, by hashing generations and looking for a repeat. Under noise a repeat would not mean a cycle, because the rule is no longer the only thing changing the board, so the detector is disabled and the board reports itself as perturbed instead. For the same reason an empty board no longer pauses the simulation when noise is on: with noise, extinction is temporary.

Cells the noise flipped on are painted amber rather than the usual white-for-newly-born, which turned out to matter more than expected. Watching a single amber cell appear next to a block and then watching the block turn into something else is the whole of Section 5.4 in one image.

5.2 Method

Four experiments, all on a torus, all with a seeded generator.

1. **Steady state against noise rate.** A 96×96 soup at 30% density is run for 700 generations of burn-in and 700 of measurement, and the density and the activity are averaged over the second half. Flips injected by the noise are not counted as activity, so activity measures what the rule is doing rather than what was put in.

2. **Structure lifetimes.** One structure is placed alone on a 32×32 torus with noise. A noise-free copy runs in lockstep and defines what the structure should look like at every generation, which handles oscillators and spaceships without special cases: the structure is intact while the noisy board agrees with the reference everywhere within two cells of the reference structure. Damage that the rule repairs within four generations does not count, so what is timed is damage that sticks.

3. **Which single flips are fatal.** The same question without the statistics: every cell within two of a structure is flipped once, on its own, with no other noise, and the board is run for 96 generations to see whether the structure comes back.

Figure 4: Conway's Life at $p = 2 \times 10^{-3}$, generation 709. Amber cells are the 32 the noise flipped this step, white cells were born by the rule, teal cells survived. The board is 7.7% full against the 2.8% it settles to with no noise, and the sparkline at the bottom right is flat rather than decaying: the board never settles.

4. **Damage spreading.** Two boards that differ in exactly one cell are driven by *identical* noise for 400 generations after a 600-generation burn-in. Sharing the noise is the point: any difference left at the end is caused by the one flip that separates the boards, not by the perturbations themselves, so the final Hamming distance says whether the noisy steady state absorbs a disturbance or amplifies it.

5.3 Result 1: Life is fed by noise; the searched rule is eroded by it

There is no lower threshold: Life's frozen state is only metastable. One flip every eleven generations, $p = 10^{-5}$ on this board, is enough to take Conway from 3.40% density and 1.31% activity to 4.44% and 2.44%. There is no rate small enough to be ignored, because the noise-free end state is not stable, only motionless: a board of still lifes stays put because nothing disturbs it, and a single disturbed cell restarts local activity that then never stops. Whatever threshold exists is at the top of the range, not the bottom.

Two rules that look alike respond in opposite directions. `B358/S126`, the rule from Task 3 that reproduces Conway's attractor more tightly than Conway does, is *destroyed* by the same noise that feeds Conway: its density falls from 2.22% to 0.13% by $p = 3 \times 10^{-4}$, seventeen times emptier, before the noise eventually overwhelms everything at $p = 10^{-2}$. Conway's ash gets denser under noise and this rule's ash gets thinner. Its stable objects are mostly 2-cell dominoes, which are the smallest thing that can persist at all, and a domino has no margin: any neighbour it acquires puts its cells outside the survival counts. Conway's block is bigger and can absorb more before it stops being a block. Two rules with the same steady state can have completely different relationships with perturbation, and nothing in the Task 3 measurements would have predicted which.

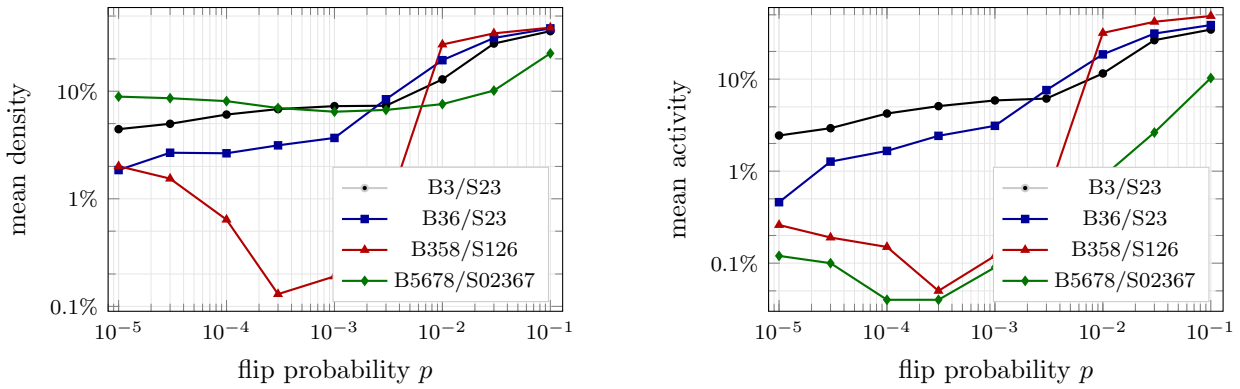


Figure 5: Steady state against noise rate, 96×96 torus, averaged over the last 700 of 1400 generations and three seeds. With no noise the four rules sit at 3.40%, 2.65%, 2.22% and 9.02% density and 1.31%, 0.61%, 0.23% and 0.13% activity.

Chaotic and saturating rules do not notice. `B25/S2468` sits at 39.2% density and 40.6% activity at every noise rate from 0 to 10^{-2}, changing in the third significant figure. Noise is invisible against a system that is already generating disorder faster than the noise supplies it. Sensitivity to perturbation is a property of *ordered* states.

5.4 Result 2: Life almost never repairs damage

The clearest measurement in the project takes no statistics at all. For each structure, every cell within two of it was flipped once, on its own, and the board run for 96 generations to see whether the structure came back.

structure	cells	neighbourhood	fatal flips	fatal	predicted lifetime	measured
blinker	3	35	25	71%	40	47
tub	4	45	25	56%	40	45
block	4	36	28	78%	36	44
boat	5	46	33	72%	30	39
glider	5	46	33	72%	30	31
beehive	6	52	38	73%	26	32
toad	6	46	34	74%	29	30
loaf	7	58	43	74%	23	28
pond	8	60	52	87%	19	23
beacon	8	56	50	89%	20	24
lightweight spaceship	9	70	52	74%	19	25

Table 4: Single-flip census under `B3/S23`. "Predicted lifetime" is $1/(p\,F)$ at $p = 10^{-3}$, where F is the number of fatal flips; "measured" is the mean over 40 noisy runs. Lifetimes are in generations.

- **Between 56% and 89% of single flips near a structure destroy it permanently.** Life has no repair mechanism. A structure is stable in the sense that it does not change on its own, not in the sense that it returns after being pushed.
- **The one-flip census predicts the noisy lifetimes.** Taking $1/(pF)$ as the expected time until a fatal flip lands reproduces the measured mean lifetime to within 25% for every structure, and to within a few percent for several. The mechanism is simply the first fatal flip. The

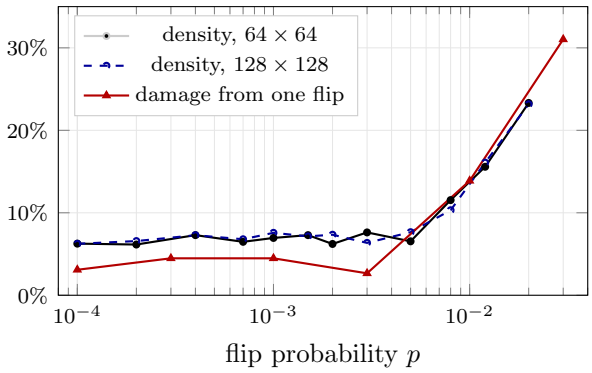
measured lifetimes are slightly longer than predicted throughout, which is what should happen given that a flip is only counted as damage once it has persisted four generations.

- **Robustness is almost entirely size.** Ranking the structures by lifetime gives blinker, tub, block, boat, glider, beehive, toad, loaf, pond, beacon, lightweight spaceship, which is very nearly the ranking by the size of their neighbourhood, smallest first. A big structure presents more surface to be hit. The pond, 8 cells, survives half as long as the blinker, 3 cells.
- **Big structures are much worse off than this table suggests.** At $p = 3 \times 10^{-4}$ the pulsar lasts 15 generations, the Gosper glider gun 13, and the HighLife replicator 26. The gun is 36 cells and its neighbourhood is nearly 300; that a machine capable of unbounded growth is destroyed in 13 generations by three flips per generation somewhere on a 96×96 board is the sharpest statement of the fragility here.
- **Moving does not help.** A glider is no more robust than a still life of the same size, and the lightweight spaceship is among the least robust things measured. A spaceship sweeps new territory, so it meets fresh noise rather than escaping it.

Two smaller observations. Under `B25/S2468` and `B2456/S156` the same block and blinker last 7 to 11 generations rather than 44 to 47: in a chaotic or saturating rule, the structure is destroyed by the board's own products before the noise gets to it. And Conway and HighLife give *identical* lifetimes, 44 and 47, because neither the structures nor the sparse debris around them ever presents a cell with six live neighbours, so at low density the two rules are the same rule.

5.5 Result 3: where the behaviour changes character

Three independent measurements were used to look for a threshold, and they agree.



p	healed	mean damage
0	15/20	1.74%
3×10^{-5}	17/20	1.27%
10^{-4}	14/20	3.09%
3×10^{-4}	13/20	4.48%
10^{-3}	14/20	4.48%
3×10^{-3}	16/20	2.66%
10^{-2}	**8/20**	**13.88%**
3×10^{-2}	**4/20**	**31.03%**

Table 5: Two boards differing in one cell, driven by identical noise.

Figure 6: Conway across the transition. Density plateaus near 7% up to $p \approx 5 \times 10^{-3}$ and then climbs; the damage from a single flip is contained below that and spreads above it. The two board sizes agree throughout, so the change is not a finite-size artefact.

1. **Density plateaus and then climbs.** From $p = 10^{-4}$ to $p = 5 \times 10^{-3}$, a factor of fifty, Conway's steady-state density stays between 6.2% and 7.6%, which is within the scatter of the measurement. Above that it climbs steadily: 11.5% at 8×10^{-3}, 15.6% at 1.2×10^{-2}, 23.3% at 2×10^{-2}.
2. **Activity does the same thing.** It tracks the density closely, which is itself informative: above the knee the board is not a denser ash, it is a boiling state where density and activity are the same number.

3. **Damage from a single flip stops being contained.** Below 3×10^{-3}, 13 to 17 of every 20 single-cell perturbations heal completely even though the noise keeps running, and the mean surviving damage is 1% to 4% of the board. At 10^{-2} only 8 of 20 heal and the damage is 14% of the board; at 3×10^{-2}, 4 of 20 and 31%.

So there is a threshold, at $p \approx 6 \times 10^{-3}$, but it is a crossover rather than a sharp transition: the curves bend over a factor of two or three in p rather than jumping, and doubling the board size does not sharpen them, which is what would be expected of a genuine phase transition. Calling it a phase transition would be overclaiming from this data.

There is a simple reading of where the number comes from. The mean time until a fatal flip lands in a given structure's neighbourhood is about $1/(pF)$, roughly $1/30p$ generations. At $p = 6 \times 10^{-3}$ that is about 6 generations, which is the same order as the time a small disturbance takes to settle down into new stable debris, a few tens of generations. Below the threshold, structures are destroyed more slowly than the board can rebuild them, and an ash exists in the steady state. Above it, they are destroyed faster than they can form, and there is no ash to disturb. This is consistent with the numbers rather than derived from them, but it does predict the right order of magnitude, and it explains why the threshold sits where it does rather than anywhere else.

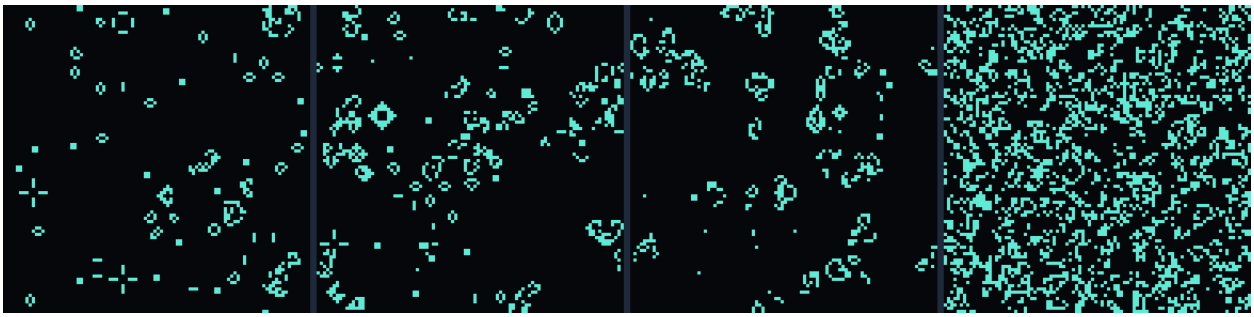


Figure 7: Conway's Life after 1200 generations at $p = 0$, 3×10^{-4}, 3×10^{-3} and 3×10^{-2}. The first three straddle a factor of ten in noise and look much the same, which is the plateau; the fourth is on the other side of the threshold and has no structures left in it at all.

5.6 Result 4: one flip on a settled board

Finally, the noise-free case, which is what the *Flip one cell* button on the site does. Sixty Conway soups were run until the board entered a cycle, one cell was flipped, and the flipped board was compared with the untouched one for 400 generations.

52 of 60 healed completely, meaning the two boards became identical again. In the 8 that did not, the difference peaked at 81 cells on average. Settled Conway ash is therefore mostly, but not reliably, insensitive to a single perturbation, and the failures are not small: when a flip does survive, it typically restarts a region rather than leaving a small scar. That combination, usually nothing and occasionally a lot, is the same shape as the fatal-flip census: most of the board is empty, a flip in empty space dies immediately, and a flip that lands near a structure destroys it about three quarters of the time.

6 How I used AI

I used Claude Code, Anthropic's command-line agent, throughout the project, and it wrote most of the code. The division of labour was roughly this.

- **Implementation.** The engine, the React interface, the rule editor, the noise controls, and all of the experiment scripts were written by the agent from prompts describing what I wanted. I reviewed the code, asked for changes when it was more complicated than the problem needed, and read through anything I did not understand until I did.
- **Experiment design.** This was the most useful part and the most collaborative. I would say what I wanted to know, for example "which patterns are robust under noise", and we would go back and forth on what would actually measure that. Several of the definitions in this report came out of those exchanges: the idea of running a noise-free reference board in lockstep to define what a structure "should" look like, and the idea of driving two boards with identical noise so that any surviving difference is attributable to the single flip that separates them.
- **Debugging.** Two of the more useful moments were failures. The pattern catalogue is tested by asserting the behaviour each description claims, and that caught a "replicator" that simply died after 5 generations. Rather than guess again, we enumerated every pattern in a 5×5 box with 5, 6 or 7 cells, about 470,000 of them, and ran each for 24 generations looking for one that becomes two translated copies of itself. There were none, which proved the replicator had to be larger and ruled out a whole class of guesses at once. Separately, a bug in a test helper made the pulsar and the Gosper gun look broken when both were correct, which was a good reminder that the measuring instrument is part of what needs checking.
- **Learning.** I used it to explain things I had not met before: outer-totalistic rule notation, why damage spreading is the standard way to ask whether a perturbation is amplified, and what a finite-size check is for.
- **Making this Writeup.** I used it to summarize the results of this investigation into a well-formatted .tex file and generate the graphics and figures.

However, most of the experimentation was driven by myself with Claude as a secondary guide. I personally observed the simulations and classified the rulesets myself.

7 Limitations

- **Board size and run length.** Everything here runs on boards between 32×32 and 128×128 for at most a few thousand generations. Conway soups on a 96×96 torus routinely take 1200 to 1800 generations to settle, so the boards are only about one order of magnitude larger than the structures they contain. Some of what I call a steady state may be a finite-size effect.
- **Classification by thresholds.** The rule classes in Section 4 are cuts through two measured numbers. They are reproducible and stated explicitly, but the boundaries are mine, and a rule sitting near one of them could be labelled either way. The measurement that a rule can change class depending on its initial density makes this concrete.
- **The search is not a proof.** Screening 3000 rules found candidates that satisfy two properties Conway has. That is not the same as finding a rule that is interesting in the way Conway is interesting, which involves things I did not measure at all, such as whether the rule supports guns, computation, or stable long-range signalling.
- **Noise model.** I used one noise model: each cell flips independently with the same probability at every step. Noise that only turns cells on, or only off, or that is correlated in space, would probably behave differently, and the asymmetry matters because a sparse board has far more dead cells than live ones, so symmetric flipping is mostly an injection of new life.

- **Statistics.** Most points here are 3 to 40 trials. That is enough to see effects of a factor of two but not enough to pin down an exponent, and the non-monotonic wobble in the finite-size table is sampling noise rather than structure.

8 General thoughts

The result that surprised me most was from Task 1 and I keep coming back to it: a board that starts 10% full and a board that starts 70% full both end up at about 3% full. Nothing in `B3/S23` mentions densities, or regions, or boundaries. The rule only ever counts to eight. The fact that a fixed point exists at all, and that it is the same fixed point from almost any starting condition, is the clearest thing in the project, and it is the reason the phrase "self-organisation" stopped being an abstraction for me.

The second thing that stuck was how rare interesting rules are. Roughly 85% of a random sample either fills the board or boils, half the rule space is spoiled by a single bit, and after screening 3000 rules only about 2% were even worth measuring properly. Conway's rule is not a typical point in its own rule space. Whether that is a fact about cellular automata or a fact about what humans find interesting is a question I would like to think about more.

The third is from Task 4: Life does not repair itself. Almost every flip that lands within two cells of a structure destroys that structure, and what looks like stability is really the absence of anything trying to knock it over. That is a real difference from biological systems, which repair damage actively, and it makes me want to know whether there are rules in this space that do repair, and what a rule would have to look like for repair to be possible at all.

What I would like to learn more about, in order: larger neighbourhoods and continuous states, since Lenia and SmoothLife seem to get much more organism-like behaviour out of the same idea; how to search rule spaces properly rather than with the hand-built filter I used here; and whether anything in this space can be made to undergo selection rather than just reproduction, because the HighLife replicator copies itself perfectly and that turns out to be exactly the wrong property for evolution.

9 Reproducing this

```
npm install
npm test                          # 31 engine and pattern tests
npm run dev                       # the site, at http://localhost:3000
node scripts/survey.mjs           # Tasks 1 and 2  -> docs/survey-output.md
node scripts/rule-survey.mjs      # Task 3         -> docs/rule-survey-output.md
node scripts/noise-survey.mjs     # Task 4         -> docs/noise-output.md
```

On the site, the two headline observations can be reproduced by hand. For Task 1, set the density slider anywhere between 10% and 70%, press Randomize, then Play at 120 generations per second and watch the population sparkline flatten at roughly the same level whatever density you started from. For Task 4, do the same and then raise the noise slider one notch at a time: at $p = 10^{-4}$ the ash keeps being disturbed and rebuilt, and at $p = 10^{-2}$ there is no ash left to disturb.